Collected Interview Questions

1. Python Interview Questions - Theory 4
1.1. Define iterators in Python 4
1.2. Define generator object in Python 4
1.3. Define scope in Python 5
1.4. Define decorators in Python 5
1.5. Explain SOLID principles in Python 6
1.6. Explain multithreading and multiprocessing in Python and the difference between them 8
1.7. Explain GIL in Python 9
1.8. Explain typing in Python 10
1.9. Explain Context Managers in Python 10
1.10. Explain class and static methods in Python 11
1.11. Explain abstract classes in Python 12
1.12. Explain the difference between shallow and deep copy in Python 12
1.13. Explain the difference between del and remove() on lists in Python 13
1.14. Explain lambda functions in Python and when to use them 13
1.15. What is the difference between lists, tuples, and sets in Python 14
1.16. What is indentation in Python, and why is it important 14
1.17. What is inheritance in Python? 15
1.18. Explain multiple inheritance in Python and how it works 15
1.19. What does * and ** represent when put before arguments 16
1.20. Can you explain common searching and graph traversal algorithms in Python? 16
1.21. Can you explain monkey patching? 17
1.22. Can you explain hidden methods in Python classes? 18
1.23. What is __init__() in Python? 19
1.24. Explain list, dictionary, and tuple comprehension with an example 20
1.25. What is a KeyError in Python, and how can you handle it? 20
1.26. How does Python handle memory management, and what role does garbage collection play? 21
1.27. Why use else in a try/except construct in Python? 21
1.28. What are the advantages of NumPy over regular Python lists? 22
1.29. What is the difference between merge, join, and concatenate in pandas? 22
1.30. How do you identify and deal with missing values in pandas? 23
1.31. Which Python libraries have you used for visualization 24
1.32. What is PEP8 and why is it important? 25
1.33. What are some common design patterns used in Python? 26
1.34. Explain how to execute SQL queries using Python 26
1.35. Explain asynchronous programming in Python 27
1.36. What is the purpose of docstrings in Python? 27
1.37. What is unit testing and how do you perform it in Python? 28
1.38. Explain map, filter, and reduce functions in Python 29
1.39. Explain the concept of metaclasses in Python 30

1. Python Interview Questions - Theory

1.1. Define iterators in Python

In Python, an **iterator** is an object that can be iterated upon (i.e., you can traverse through the values it holds). Formally, an iterator must implement two special methods:

`__iter__()`: Returns the iterator object itself.

`__next__()`: Returns the next item from the sequence and raises a `StopIteration` exception when there are no more items.

Any object that implements these methods (or uses the built-in `iter()` and `next()` functions) can be used in a `for` loop or other iteration contexts in Python.

1.2. Define generator object in Python

A **generator object** in Python is an iterator-like object created by a **generator function** or a **generator expression**.

Generator Function: Any function that contains one or more `yield` statements. When called, it doesn't execute all its code at once; instead, it returns a generator object. Each call to the generator's `__next__()` (or using `next()`) resumes the function right after the last `yield`.

Generator Expression: Similar to a list comprehension, but uses parentheses (`(...)`) instead of square brackets (`[...]`). When evaluated, it returns a generator object.

Key benefits of using generators include:

Lazy evaluation: They produce values on the fly, making them memory-efficient for large datasets.

State preservation: The internal state of the generator is preserved between `yield` calls, allowing the function to resume where it left off.

1.3. Define scope in Python.

In Python, **scope** refers to the region or context in a program where a variable or function is accessible. Python uses the LEGB rule to determine the scope of a variable:

Local: Variables defined inside a function.

Enclosing: Variables in the local scope of enclosing functions (e.g., nested functions).

Global: Variables defined at the top level of a script or declared as `global` within a function.

Built-in: Names pre-defined in Python (e.g., `len`, `range`).

The scope determines the lifetime and visibility of variables and helps prevent conflicts between variable names in different parts of a program.

1.4. Define decorators in Python

In Python, a **decorator** is a design pattern that allows you to modify the behavior of a function or class method. Decorators are functions that take another function as an argument, extend or modify its behavior, and return the modified function. They are often used to add functionality in a reusable and concise way.

Syntax: Decorators are applied to functions using the `@decorator_name` syntax, placed above the function definition.

```
def decorator_function(original_function):
    def wrapper_function(*args, **kwargs):
        print("Wrapper executed before", original_function.__name__)
        return original_function(*args, **kwargs)
    return wrapper_function
```

```
@decorator_function
def display():
    print("Display function executed")

display()
```

Common use cases:

- Logging
- Access control and authentication
- Caching
- Timing functions

1.5. Explain SOLID principles in Python

The **SOLID principles** are a set of design principles aimed at making software designs more maintainable and scalable. These principles, though language-agnostic, are widely used in Python for creating robust code:

S - Single Responsibility Principle (SRP): A class or function should have only one reason to change, meaning it should have only one responsibility. This helps in keeping the code modular and easier to maintain.

Example:

```
class Invoice:
    def calculate_total(self):
        # Calculate invoice total
        pass

class InvoicePrinter:
    def print_invoice(self):
        # Print invoice details
        pass
```

O - Open/Closed Principle (OCP): Software entities (classes, functions, modules) should be open for extension but closed for modification. This can be achieved using inheritance or polymorphism. **Example**:

```
 class Shape:
    def area(self):
        pass

class Circle(Shape):
    def area(self):
        return 3.14 * self.radius ** 2
```

L - Liskov Substitution Principle (LSP): Subtypes should be substitutable for their base types without altering the correctness of the program. **Example**:

```
 class Bird:
    def fly(self):
        pass

class Sparrow(Bird):
    def fly(self):
        print("Sparrow flying")
```

I - Interface Segregation Principle (ISP): A class should not be forced to implement interfaces it doesn't use. In Python, this can be handled by creating smaller, more specific interfaces (or abstract base classes). **Example**:

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def eat(self):
        pass

class FlyingAnimal(Animal):
    @abstractmethod
    def fly(self):
        pass

class Bird(FlyingAnimal):
    def eat(self):
        print("Bird eating")

    def fly(self):
        print("Bird flying")
```

D - Dependency Inversion Principle (DIP): High-level modules should not depend on low-level modules. Both should depend on abstractions. This can be achieved using dependency injection. **Example**:

```
 class Database:
    def connect(self):
        pass

class MySQLDatabase(Database):
    def connect(self):
        print("Connecting to MySQL")

class Application:
    def __init__(self, database: Database):
        self.database = database

    def start(self):
        self.database.connect()

db = MySQLDatabase()
app = Application(db)
app.start()
```

These principles promote modularity, reusability, and robustness in software development.

1.6. Explain multithreading and multiprocessing in Python and the difference between them

Multithreading: It involves running multiple threads within a single process. Threads share the same memory space, making communication between them easier, but they are limited by Python's Global Interpreter Lock (GIL). **Example**:

```
 import threading

def print_numbers():
    for i in range(5):
        print(i)

thread = threading.Thread(target=print_numbers)
thread.start()
thread.join()
```

Multiprocessing: It involves running multiple processes, each with its own memory space. This is more effective for CPU-bound tasks as it bypasses the GIL. **Example**:

```
 from multiprocessing import Process

def print_numbers():
    for i in range(5):
        print(i)

process = Process(target=print_numbers)
process.start()
process.join()
```

Differences:

Memory: Threads share memory; processes have separate memory.

Overhead: Threads have lower overhead; processes are more resource-intensive.

GIL: Threads are limited by the GIL in Python; processes are not.

Use case: Use threads for I/O-bound tasks; use processes for CPU-bound tasks.

1.7. Explain GIL in Python

The **Global Interpreter Lock (GIL)** is a mutex that protects access to Python objects, preventing multiple native threads from executing Python bytecodes simultaneously in a multi-threaded program. This ensures thread safety but can be a bottleneck for CPU-bound tasks as only one thread executes at a time.

Key points:

The GIL affects only CPython.

It allows only one thread to execute Python code at a time, even on multi-core processors.

It does not affect multiprocessing.

Workarounds:

Use multiprocessing for CPU-bound tasks.

Use libraries like `numpy` that release the GIL during computation.

1.8. Explain typing in Python

Typing in Python refers to type annotations that help developers specify the expected data types of variables, function parameters, and return values. Python's typing module enables static type checking, improving code readability and maintainability. **Example**:

```
def add_numbers(a: int, b: int) -> int:
    return a + b
```

Common types:

`int`, `float`, `str`, `bool`

`List`, `Tuple`, `Dict`, `Set`

`Optional` for nullable types

`Union` for multiple types

Benefits: Improves documentation, catches type errors during development, and integrates with tools like MyPy for type checking.

1.9. Explain Context Managers in Python

A **context manager** in Python is used to manage resources, ensuring that they are properly acquired and released. Context managers are often used with the `with` statement. **Example**:

```
with open("file.txt", "r") as file:
    data = file.read()
# The file is automatically closed after the block
# To create a custom context manager, use the __enter__ and __exit__
methods:

 class MyContextManager:
    def __enter__(self):
```

```
        print("Entering context")

    def __exit__(self, exc_type, exc_value, traceback):
        print("Exiting context")

with MyContextManager():
    print("Inside context")
```

Benefits: Simplifies resource management and ensures proper cleanup.

1.10. Explain class and static methods in Python

Class methods: Methods bound to the class, not instances. They take the class as the first parameter (`cls`) and can modify class-level data. **Example**:

```
 class MyClass:
    count = 0

    @classmethod
    def increment_count(cls):
        cls.count += 1

MyClass.increment_count()
print(MyClass.count)  # Output: 1
```

Static methods: Methods that don't depend on instance or class-level data. They behave like regular functions but belong to the class namespace. **Example**:

```
class MyClass:
    @staticmethod
    def greet():
        return "Hello"

print(MyClass.greet())  # Output: Hello
```

Key differences:

Class methods have access to the class (`cls`) and can modify its state.

Static methods are utility functions that don't rely on class or instance data.

1.11. Explain abstract classes in Python

An **abstract class** in Python is a blueprint for other classes. It can define methods that must be implemented in any subclass. Abstract classes are created using the `ABC` (Abstract Base Class) module in the `abc` package. Abstract methods defined in these classes cannot be instantiated directly. **Example**:

```
 from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

class Dog(Animal):
    def make_sound(self):
        return "Bark"

class Cat(Animal):
    def make_sound(self):
        return "Meow"

dog = Dog()
print(dog.make_sound())  # Output: Bark
```

Abstract classes are useful for defining common interfaces for a set of subclasses.

1.12. Explain the difference between shallow and deep copy in Python

Shallow copy: Creates a new object but copies only the references of the objects inside it. Changes made to the nested objects in the copied object will reflect in the original object.
Example:

```
 import copy

original_list = [[1, 2], [3, 4]]
shallow_copied_list = copy.copy(original_list)
```

```
shallow_copied_list[0][0] = 100
print(original_list)  # Output: [[100, 2], [3, 4]]
```

Deep copy: Creates a new object and recursively copies all objects inside it. Changes to nested objects in the copied object do not affect the original object. **Example**:

```
import copy

original_list = [[1, 2], [3, 4]]
deep_copied_list = copy.deepcopy(original_list)

deep_copied_list[0][0] = 100
print(original_list)  # Output: [[1, 2], [3, 4]]
```

1.13. Explain the difference between `del` and `remove()` on lists in Python

`del`: Deletes an item at a specified index or deletes the entire list. **Example**:

```
my_list = [1, 2, 3]
del my_list[1]  # Deletes the item at index 1
print(my_list)  # Output: [1, 3]
```

`remove()`: Removes the first occurrence of a specified value from the list. **Example**:

```
my_list = [1, 2, 3, 2]
my_list.remove(2)  # Removes the first occurrence of 2
print(my_list)  # Output: [1, 3, 2]
```

1.14. Explain lambda functions in Python and when to use them

A **lambda function** in Python is a small anonymous function defined using the `lambda` keyword. It can have any number of arguments but only one expression. **Example**:

```
square = lambda x: x ** 2
print(square(5))  # Output: 25
```

Use cases:

A. When you need a simple function for a short period.
B. To pass as an argument to higher-order functions like `map()`, `filter()`, or `sorted()`.

1.15. What is the difference between lists, tuples, and sets in Python

List: Ordered, mutable, allows duplicate elements. **Example**:
my_list = [1, 2, 3, 2]

Tuple: Ordered, immutable, allows duplicate elements. **Example**:
my_tuple = (1, 2, 3, 2)

Set: Unordered, mutable, does not allow duplicate elements. **Example**:
my_set = {1, 2, 3, 2} # Output: {1, 2, 3}

1.16. What is indentation in Python, and why is it important

In Python, **indentation** is used to define blocks of code. Unlike other languages that use braces, Python uses indentation to indicate a block of code for functions, loops, and conditionals. **Example**:

```
if True:
    print("This is indented")
```

Importance:

- Ensures code readability.
- Mandatory for proper code execution; incorrect indentation will result in an `IndentationError`.

1.17.What is inheritance in Python?

Inheritance in Python allows a class (child class) to inherit the attributes and methods of another class (parent class), promoting code reuse and extensibility. **Example**:

```
class Parent:
   def greet(self):
       print("Hello from Parent")

class Child(Parent):
    pass

child = Child()
child.greet()  # Output: Hello from Parent
```

1.18.Explain multiple inheritance in Python and how it works

Multiple inheritance allows a class to inherit attributes and methods from more than one parent class. Python uses the C3 linearization (MRO - Method Resolution Order) to determine the order in which classes are searched for methods and attributes. **Example**:

```
class A:
    def greet(self):
        print("Hello from A")

class B:
    def greet(self):
        print("Hello from B")

class C(A, B):
    pass

c = C()
c.greet()  # Output: Hello from A (A is searched first due to MRO)
```

1.19. What does * and ** represent when put before arguments

`*args`: Used to pass a variable number of positional arguments to a function. **Example**:

```
def sum_all(*args):
    return sum(args)

print(sum_all(1, 2, 3))  # Output: 6
```

`**kwargs`: Used to pass a variable number of keyword arguments to a function. **Example**:

```
def print_details(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_details(name="Alice", age=25)
```

1.20. Can you explain common searching and graph traversal algorithms in Python?

Binary Search: Efficient algorithm for searching in a sorted list. **Example**:

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Depth-First Search (DFS): Traverses a graph/tree by exploring as far as possible along each branch. **Example**:

```
def dfs(graph, start, visited=set()):
    if start not in visited:
        print(start)
        visited.add(start)
        for neighbor in graph[start]:
            dfs(graph, neighbor, visited)
```

Breadth-First Search (BFS): Traverses a graph/tree level by level. **Example**:

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node)
            visited.add(node)
            queue.extend(graph[node])
```

1.21. Can you explain monkey patching?

Monkey patching is the practice of dynamically modifying or extending a class or module at runtime. It is often used to add or modify behavior without changing the original source code. While it can be useful, it should be used cautiously as it can lead to unpredictable behavior.

```
class MyClass:
    def greet(self):
        return "Hello"

# Monkey patching the greet method
def new_greet(self):
    return "Hi there!"

MyClass.greet = new_greet
```

```
obj = MyClass()
print(obj.greet())  # Output: Hi there!
```

Use cases: Quick fixes or temporary changes during testing or debugging.

Drawbacks: Reduces code maintainability and can introduce subtle bugs.

1.22. Can you explain hidden methods in Python classes?

Hidden methods in Python classes are methods that are intended to be private to the class and not accessed directly outside of it. These methods are prefixed with double underscores (e.g., `__method`). Python applies name mangling to such methods to make them harder to access from outside the class. However, they can still be accessed indirectly using the mangled name.

```
class MyClass:
    def __hidden_method(self):
        return "This is a hidden method"
    def public_method(self):
        return self.__hidden_method()
obj = MyClass()
print(obj.public_method())  # Output: This is a hidden method
# print(obj.__hidden_method())  # Raises AttributeError
print(obj._MyClass__hidden_method())  # Accessing hidden method via name mangling
```

Use case: Hidden methods are useful for encapsulation, where internal functionality is abstracted away from external access.

Question: Explain `__str__`, `__repr__`, and similar double underscore methods in Python classes.

Double underscore methods (also known as dunder methods or magic methods) in Python provide a way to define or customize the behavior of objects for built-in operations. Two commonly used dunder methods are:

- **`__str__()`**: Returns a human-readable string representation of an object. Used by the `print()` function.

- `__repr__()`: Returns an unambiguous string representation of an object, ideally including all necessary information to recreate the object. Used by the `repr()` function or interactive shells.

```
class MyClass:
    def __init__(self, value):
        self.value = value
    def __str__(self):
        return f"MyClass with value: {self.value}"
    def __repr__(self):
        return f"MyClass({self.value})"

obj = MyClass(10)
print(obj)         # Output: MyClass with value: 10
print(repr(obj))   # Output: MyClass(10
```

Other common dunder methods:

- `__add__`: Customize behavior for the `+` operator.
- `__len__`: Define behavior for the `len()` function.
- `__getitem__`: Define behavior for indexing operations.

1.23. What is `__init__()` in Python?

The `__init__()` method in Python is a special method called a constructor. It is automatically invoked when an object of a class is created. The primary purpose of `__init__()` is to initialize an object's attributes. Example:

```
class MyClass:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"Hello, my name is {self.name} and I am {self.age} years
old."

obj = MyClass("Alice", 30)
print(obj.greet())  # Output: Hello, my name is Alice and I am 30 years
old.
```

Key points:

- __init__() is not a mandatory method but is commonly used for initialization.
- It does not return any value (None is returned by default).
- It allows passing arguments during object creation to set up the initial state of the object.

1.24. Explain list, dictionary, and tuple comprehension with an example

Comprehensions in Python provide a concise way to create lists, dictionaries, or tuples by iterating over an iterable and applying an expression.

List comprehension:

```
numbers = [1, 2, 3, 4]
squares = [x**2 for x in numbers]
print(squares)  # Output: [1, 4, 9, 16]
```

Dictionary comprehension:

```
numbers = [1, 2, 3, 4]
squares_dict = {x: x**2 for x in numbers}
print(squares_dict)  # Output: {1: 1, 2: 4, 3: 9, 4: 1}
```

Tuple comprehension: Tuple comprehension does not exist as a standalone construct, but you can use generator expressions and convert them to a tuple. **Example**:

```
numbers = (1, 2, 3, 4)
squares_tuple = tuple(x**2 for x in numbers)
print(squares_tuple)  # Output: (1, 4, 9, 16)
```

1.25. What is a KeyError in Python, and how can you handle it?

A **KeyError** occurs in Python when you try to access a key in a dictionary that does not exist.

```
my_dict = {"a": 1, "b": 2}
# print(my_dict["c"])  # Raises KeyError
```

Handling KeyError:

Use the `get()` method, which returns `None` or a default value if the key is not found. **Example**:

```
print(my_dict.get("c", "Key not found"))  # Output: Key not found
```

Use `try-except`:

```
try:
    value = my_dict["c"]
except KeyError:
    print("Key not found")
```

1.26. How does Python handle memory management, and what role does garbage collection play?

Python uses automatic memory management to allocate and deallocate memory for objects. It relies on reference counting and garbage collection.

Reference counting: Each object has a reference count that tracks how many references point to it. When the count drops to zero, the memory is deallocated.

Garbage collection: Handles circular references (e.g., two objects referencing each other). Python's `gc` module detects and cleans these unused objects.

```
import gc
gc.collect()  # Manually triggers garbage collection
```

1.27. Why use `else` in a `try/except` construct in Python?

The `else` block in a `try/except` construct is executed only if no exception is raised in the `try`

block. It allows you to separate error handling from the normal execution flow.

```
 try:
    result = 10 / 2
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Division successful, result:", result)

# Output:
# Division successful, result: 5.0
```

1.28. What are the advantages of NumPy over regular Python lists?

- **Performance**: NumPy arrays are faster than Python lists due to their fixed size and contiguous memory allocation.
- **Memory efficiency**: Arrays use less memory compared to lists.
- **Vectorized operations**: Perform element-wise operations without explicit loops.
- **Rich functionality**: Provides mathematical and statistical functions.

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr * 2)  # Output: [2 4 6 8]
```

1.29. What is the difference between `merge`, `join`, and `concatenate` in pandas?

merge: Combines DataFrames using a database-style join based on common columns or indices. Highly customizable for inner, outer, left, and right joins. **Example**:

```
import pandas as pd
```

```
df1 = pd.DataFrame({'key': [1, 2], 'value': ['A', 'B']})
df2 = pd.DataFrame({'key': [2, 3], 'value': ['C', 'D']})

result = pd.merge(df1, df2, on='key', how='inner')
print(result)
```

join: Combines DataFrames based on their indices. Useful for aligning DataFrames with different indices. **Example**:

```
df1 = pd.DataFrame({'value1': [1, 2]}, index=['A', 'B'])
df2 = pd.DataFrame({'value2': [3, 4]}, index=['A', 'C'])

result = df1.join(df2, how='outer')
print(result)
```

concatenate: Stacks DataFrames either vertically (by rows) or horizontally (by columns).

```
df1 = pd.DataFrame({'A': [1, 2]})
df2 = pd.DataFrame({'A': [3, 4]})

result = pd.concat([df1, df2], axis=0)
print(result)
```

1.30. How do you identify and deal with missing values in pandas?

In pandas, missing values are represented as `NaN`. You can identify and handle them using the following methods:

Identifying missing values:

- `isnull()`: Returns a DataFrame of the same shape with `True` for missing values.
- `notnull()`: Returns `True` for non-missing values. **Example**:

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, None], 'B': [4, None, 6]})
print(df.isnull())
```

Handling missing values:

- **Drop missing values**: Use `dropna()` to remove rows or columns with missing values.

```
df_cleaned = df.dropna()
```

- **Fill missing values**: Use `fillna()` to fill missing values with a specific value or a method (e.g., forward fill).

```
df_filled = df.fillna(0)
```

- **Impute missing values**: Use statistical techniques like mean or median to replace missing values.

```
df['A'] = df['A'].fillna(df['A'].mean())
```

1.31. Which Python libraries have you used for visualization

- **Matplotlib**: A versatile library for creating static, animated, and interactive plots.

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [4, 5, 6])
plt.show()
```

- **Seaborn**: Built on Matplotlib, it provides a high-level interface for statistical graphics.

```
import seaborn as sns
sns.barplot(x=['A', 'B', 'C'], y=[10, 20, 15])
```

- **Plotly**: For interactive, web-based visualizations.

```
import plotly.express as px
fig = px.scatter(x=[1, 2, 3], y=[4, 5, 6])
fig.show()
```

- **Bokeh**: Another library for interactive visualizations.
- **Altair**: Declarative statistical visualization library based on Vega and Vega-Lite.

1.32. What is PEP8 and why is it important?

PEP8 is the Python Enhancement Proposal that provides guidelines and best practices for writing Python code. It helps improve code readability and maintainability by enforcing a consistent coding style.

Key conventions in PEP8:

- **Indentation**: Use 4 spaces per indentation level.
- **Line length**: Limit lines to 79 characters.
- **Naming conventions**: Use `snake_case` for variables and functions, `CamelCase` for classes.
- **Spacing**: Include a single blank line between functions and methods.

Importance:

- Makes code easier to read and understand.
- Promotes consistency across projects and teams.
- Improves collaboration among developers.

1.33. What are some common design patterns used in Python?

Design patterns are reusable solutions to common software design problems. Commonly used patterns in Python include:

Singleton: Ensures a class has only one instance and provides a global point of access to it.

```
class Singleton:
    _instance = None
    def __new__(cls, *args, **kwargs):
        if not cls._instance:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Factory: Creates objects without specifying the exact class.

Observer: Notifies subscribers when an event occurs.

Decorator: Dynamically adds behavior to an object at runtime.

1.34. Explain how to execute SQL queries using Python

You can execute SQL queries in Python using libraries like `sqlite3`, `psycopg2` (for PostgreSQL), or `mysql-connector` (for MySQL).

```
import sqlite3

# Connect to a database or create one
connection = sqlite3.connect("example.db")
cursor = connection.cursor()

# Execute SQL commands
cursor.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER, name TEXT)")
cursor.execute("INSERT INTO users (id, name) VALUES (?, ?)", (1, "Alice"))

# Fetch results
cursor.execute("SELECT * FROM users")
```

```
print(cursor.fetchall())

# Commit changes and close connection
connection.commit()
connection.close()
```

1.35. Explain asynchronous programming in Python

Asynchronous programming allows Python to perform non-blocking tasks using the `asyncio` module. This is useful for I/O-bound tasks like file operations, network requests, or database queries.

Key components:

- **`async` keyword**: Marks a function as asynchronous.
- **`await` keyword**: Pauses execution until an asynchronous task completes.
- **`asyncio.run()`**: Runs an asynchronous event loop.

```
import asyncio
async def fetch_data():
    print("Fetching data...")
    await asyncio.sleep(2)  # Simulates an I/O operation
    print("Data fetched")

asyncio.run(fetch_data())
```

Benefits:

- Improves performance for I/O-bound tasks.
- Enables concurrent execution of multiple tasks.

1.36.What is the purpose of docstrings in Python?

Docstrings (documentation strings) in Python are used to document a module, class, or function. They provide a convenient way to explain the purpose and functionality of code, making it easier to understand and maintain.

Key features:

- Defined using triple quotes (`'''` or `"""`).
- Accessible using the `.__doc__` attribute.

```
def add(a, b):
    """This function returns the sum of two numbers.
    Args:
        a (int): First number.
        b (int): Second number.
    Returns:
        int: Sum of a and b.
    """
    return a + b

print(add.__doc__)
```

Benefits:

- Improves code readability.
- Serves as a quick reference for developers.
- Enables automated documentation generation using tools like Sphinx.

1.37.What is unit testing and how do you perform it in Python?

Unit testing involves testing individual components of code (like functions or methods) to ensure they work as expected. In Python, the `unittest` module provides a framework for writing and running unit tests.

Steps to perform unit testing:

- Import the `unittest` module.
- Create a test class that inherits from `unittest.TestCase`.
- Define test methods in the class. Test methods must start with `test_`.
- Use `assert` methods (e.g., `assertEqual`, `assertTrue`) to validate functionality.

```
import unittest
def add(a, b):
    return a + b
class TestMathOperations(unittest.TestCase):
```

```
    def test_add(self):
        self.assertEqual(add(2, 3), 5)
        self.assertEqual(add(-1, 1), 0)
if __name__ == '__main__':
    unittest.main()
```

Benefits:

- Detects bugs early.
- Ensures code reliability.
- Facilitates refactoring by providing immediate feedback.

1.38. Explain `map`, `filter`, and `reduce` functions in Python

These are higher-order functions in Python used for transforming and processing data:

- **`map()`**: Applies a function to each item in an iterable and returns a map object (which can be converted to a list). **Example**:

```
numbers = [1, 2, 3, 4]
squares = map(lambda x: x**2, numbers)
print(list(squares))  # Output: [1, 4, 9, 16]
```

- **`filter()`**: Filters elements of an iterable based on a function that returns `True` or `False`.

```
numbers = [1, 2, 3, 4]
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers))  # Output: [2, 4]
```

- **`reduce()`**: Applies a function cumulatively to the items of an iterable, reducing it to a single value. It's part of the `functools` module.

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
product = reduce(lambda x, y: x * y, numbers)
print(product)  # Output: 24
```

1.39. Explain the concept of metaclasses in Python

A **metaclass** in Python is a class that defines the behavior of other classes. Just as a class creates instances, a metaclass creates classes. Metaclasses allow customization of class creation and behavior.

How metaclasses work:

- By default, Python classes are instances of the `type` metaclass.
- You can define a custom metaclass by inheriting from `type`.

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        print(f"Creating class {name}")
        return super().__new__(cls, name, bases, dct)

class MyClass(metaclass=Meta):
    pass

# Output: Creating class MyClass
```

Use cases:

- Enforcing coding standards
- Automatically registering classes
- Modifying class attributes or methods during creation